Atelier 2

Procédures et fonctions stockées en MySQL

Rapport de travaux pratiques

Gestion des stagiaires et facturation de l’eau

12 novembre 2026

Sommaire

1. Introduction 3

2. Rappel du cours 3

2.1 Procédure stockée 3

2.2 Fonction stockée 3

2.3 Le délimiteur 3

2.4 Différences principales 3

3. Étude de cas : gestion des stagiaires 4

3.1 Création de la base, de la table et des données 4

3.2 Une procédure simple : pc1 5

3.3 Compter les stagiaires : compter_stagiaires 5

3.4 Afficher les stagiaires d'une ville : afficher_stagiaires_par_ville 6

3.5 Une fonction : nombre_stagiaires_par_ville 6

3.6 Lister les procédures et les fonctions 7

3.7 Suppression des procédures et de la fonction 7

4. Facturation de l'eau 8

4.1 Insertion de cent factures 8

4.2 La fonction calculer_montant 9

4.3 Tests de la fonction 9

5. Les paramètres IN, OUT et INOUT 10

5.1 Paramètre IN : supprimer_stagiaire 10

5.2 Paramètre OUT : compter_stagiaires_out 10

5.3 Paramètre INOUT : augmenter_stock 11

5.4 Récapitulatif 11

6. Conclusion 12

1. Introduction

Ce rapport présente le travail réalisé pendant l'atelier 2 de MySQL, consacré aux procédures stockées et aux fonctions stockées. L'idée est d'enregistrer des instructions SQL directement dans la base de données, pour pouvoir ensuite les relancer avec une seule commande au lieu de les réécrire à chaque fois.

Toutes les manipulations ont été faites en ligne de commande : le client `mysql` est lancé depuis Windows PowerShell et se connecte à un serveur MySQL 8.4. Le travail se fait dans une base nommée `atelier2`. Chaque étape est illustrée par une capture d'écran qui montre le code exécuté et le résultat obtenu.

Le rapport suit l'ordre de l'atelier : un petit rappel de cours, l'étude de cas sur les stagiaires, l'exemple de la facturation de l'eau, puis les trois types de paramètres (IN, OUT et INOUT).

2. Rappel du cours

2.1 Procédure stockée

Une procédure stockée est un programme enregistré dans la base. Elle regroupe plusieurs instructions SQL sous un même nom et se lance avec `CALL nom_procedure(...)`. Elle peut recevoir des paramètres, déclarer des variables avec `DECLARE` et les remplir avec `SET` ou `SELECT ... INTO`. Pour la supprimer on utilise `DROP PROCEDURE`.

2.2 Fonction stockée

Une fonction stockée ressemble à une procédure, mais elle retourne obligatoirement une valeur grâce à `RETURN`, avec le type indiqué après `RETURNS`. Comme elle renvoie une valeur, on peut l'utiliser directement dans une requête, par exemple `SELECT ma_fonction(...)`. Elle se supprime avec `DROP FUNCTION`.

2.3 Le délimiteur

Le corps d'une procédure contient des points-virgules. Si on ne fait rien, MySQL croit que l'instruction est finie dès le premier et renvoie une erreur. On change donc le délimiteur avant la création (`DELIMITER /`), puis on remet le point-virgule après le `END` (`DELIMITER ;`). Dans tout l'atelier, le caractère utilisé est la barre oblique.

2.4 Différences principales

	Procédure	Fonction
Appel	CALL nom(...)	SELECT nom(...)
Valeur retournée	Pas de RETURN (résultat affiché ou renvoyé par un paramètre OUT)	RETURN obligatoire
Paramètres	IN, OUT et INOUT	IN seulement
Suppression	DROP PROCEDURE	DROP FUNCTION

3. Étude de cas : gestion des stagiaires

3.1 Création de la base, de la table et des données

On commence par créer la base `atelier2` et on la sélectionne avec `USE`. La table `stagiaires` a trois colonnes : `id` (clé primaire en AUTO_INCREMENT), `nom` et `ville`. Cinq stagiaires sont ajoutés avec un seul `INSERT`, puis un `SELECT *` permet de vérifier que les données sont bien dans la table.

```
MySQL [defaultdb]> CREATE DATABASE IF NOT EXISTS atelier2;
Query OK, 1 row affected (0.161 sec)

MySQL [defaultdb]> USE atelier2;
Database changed
MySQL [atelier2]> CREATE TABLE stagiaires (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     nom VARCHAR(100),
    ->     ville VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.654 sec)

MySQL [atelier2]> INSERT INTO stagiaires (nom, ville) VALUES
    -> ('Ali', 'Casablanca'),
    -> ('Sara', 'Rabat'),
    -> ('Khalid', 'Casablanca'),
    -> ('Mouna', 'Marrakech'),
    -> ('Youssef', 'Rabat');
Query OK, 5 rows affected (0.125 sec)
Records: 5  Duplicates: 0  Warnings: 0

MySQL [atelier2]> SELECT * FROM stagiaires;
+----+---------+------------+
| id | nom     | ville      |
+----+---------+------------+
|  1 | Ali     | Casablanca |
|  2 | Sara    | Rabat      |
|  3 | Khalid  | Casablanca |
|  4 | Mouna   | Marrakech  |
|  5 | Youssef | Rabat      |
+----+---------+------------+
5 rows in set (0.049 sec)
```

Figure 1 : création de la base, de la table stagiaires et insertion des données

3.2 Une procédure simple : pc1

La procédure `pc1` ne prend aucun paramètre. Elle contient une seule instruction, `SELECT * FROM stagiaires`. Après l'appel avec `CALL pc1();`, on obtient les cinq lignes de la table.

```
MySQL [atelier2]> DELIMITER /
MySQL [atelier2]> CREATE PROCEDURE pc1()
    -> BEGIN
    ->     SELECT * FROM stagiaires;
    -> END /
Query OK, 0 rows affected (0.147 sec)

MySQL [atelier2]> DELIMITER ;
MySQL [atelier2]> CALL pc1();
+----+---------+------------+
| id | nom     | ville      |
+----+---------+------------+
|  1 | Ali     | Casablanca |
|  2 | Sara    | Rabat      |
|  3 | Khalid  | Casablanca |
|  4 | Mouna   | Marrakech  |
|  5 | Youssef | Rabat      |
+----+---------+------------+
5 rows in set (0.066 sec)

Query OK, 0 rows affected (0.071 sec)
```

Figure 2 : création et appel de la procédure pc1

3.3 Compter les stagiaires : compter_stagiaires

Cette procédure déclare une variable locale `total` de type INT. Elle y range le résultat de `COUNT(*)` avec `SELECT ... INTO`, puis l'affiche sous le nom « Nombre de stagiaires ». Le résultat est 5, ce qui correspond bien aux cinq lignes insérées.

```
MySQL [atelier2]> DELIMITER /
MySQL [atelier2]> CREATE PROCEDURE compter_stagiaires()
    -> BEGIN
    ->     DECLARE total INT;
    ->     SELECT COUNT(*) INTO total FROM stagiaires;
    ->     SELECT total AS 'Nombre de stagiaires';
    -> END /
Query OK, 0 rows affected (0.076 sec)

MySQL [atelier2]> DELIMITER ;
MySQL [atelier2]> CALL compter_stagiaires();
+----------------------+
| Nombre de stagiaires |
+----------------------+
|                    5 |
+----------------------+
1 row in set (0.186 sec)

Query OK, 0 rows affected (0.191 sec)
```

Figure 3 : procédure compter_stagiaires et son résultat

3.4 Afficher les stagiaires d'une ville : afficher_stagiaires_par_ville

Ici la procédure reçoit un paramètre d'entrée `ville_param` (VARCHAR(100)) qui est utilisé dans la clause `WHERE`. En l'appelant avec la valeur `'Casablanca'`, on obtient deux stagiaires : Ali et Khalid.



Figure 4 : procédure avec un paramètre IN, appelée avec la ville Casablanca

3.5 Une fonction : nombre_stagiaires_par_ville

La fonction est construite comme la procédure précédente, mais au lieu d'afficher un tableau elle retourne un entier (`RETURNS INT`). Elle compte les stagiaires de la ville donnée dans la variable `total` puis la renvoie avec `RETURN`. Comme c'est une fonction, on l'appelle dans un `SELECT` : pour Rabat elle retourne 2 (Sara et Youssef).

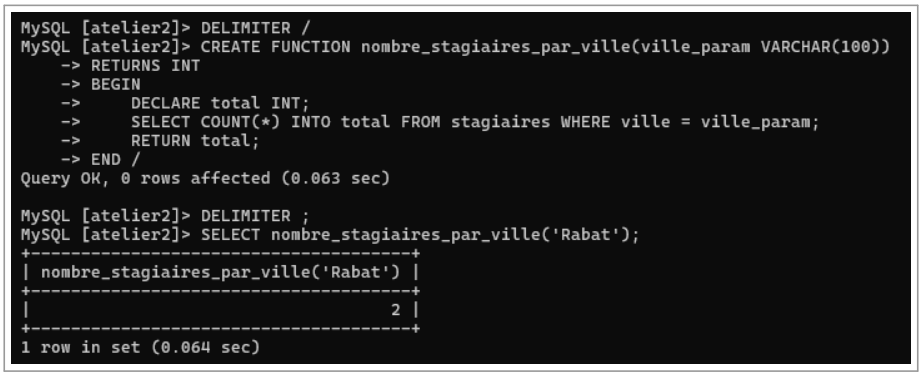


Figure 5 : création et appel de la fonction nombre_stagiaires_par_ville

3.6 Lister les procédures et les fonctions

`SHOW PROCEDURE STATUS WHERE Db = DATABASE();` affiche les procédures de la base courante. À ce stade il y en a trois : `afficher_stagiaires_par_ville`, `compter_stagiaires` et `pc1`. La commande équivalente pour les fonctions montre une seule ligne, `nombre_stagiaires_par_ville`. On voit aussi la date de création et l'utilisateur qui a créé l'objet.



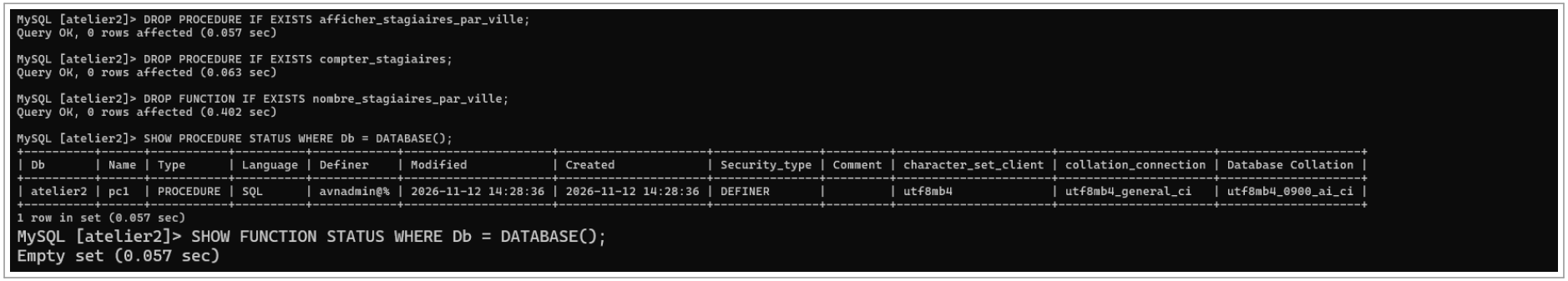
```
MySQL [atelier2]> SHOW PROCEDURE STATUS WHERE Db = DATABASE();
+----------+-------------------------------+-----------+----------+-------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
| Db       | Name                          | Type      | Language | Definer     | Modified            | Created             | Security_type | Comment | character_set_client | collation_connection | Database Collation |
+----------+-------------------------------+-----------+----------+-------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
| atelier2 | afficher_stagiaires_par_ville | PROCEDURE | SQL      | avnadmin@% | 2026-11-12 14:32:59 | 2026-11-12 14:32:59 | DEFINER       |         | utf8mb4              | utf8mb4_general_ci   | utf8mb4_0900_ai_ci |
| atelier2 | compter_stagiaires            | PROCEDURE | SQL      | avnadmin@% | 2026-11-12 14:31:23 | 2026-11-12 14:31:23 | DEFINER       |         | utf8mb4              | utf8mb4_general_ci   | utf8mb4_0900_ai_ci |
| atelier2 | pc1                           | PROCEDURE | SQL      | avnadmin@% | 2026-11-12 14:28:36 | 2026-11-12 14:28:36 | DEFINER       |         | utf8mb4              | utf8mb4_general_ci   | utf8mb4_0900_ai_ci |
+----------+-------------------------------+-----------+----------+-------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
3 rows in set (0.151 sec)

MySQL [atelier2]> SHOW FUNCTION STATUS WHERE Db = DATABASE();
+----------+-------------------------+----------+----------+-------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
| Db       | Name                    | Type     | Language | Definer     | Modified            | Created             | Security_type | Comment | character_set_client | collation_connection | Database Collation |
+----------+-------------------------+----------+----------+-------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
| atelier2 | nombre_stagiaires_par_ville | FUNCTION | SQL      | avnadmin@% | 2026-11-12 14:34:28 | 2026-11-12 14:34:28 | DEFINER       |         | utf8mb4              | utf8mb4_general_ci   | utf8mb4_0900_ai_ci |
+----------+-------------------------+----------+----------+-------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
1 row in set (0.053 sec)
```

Figure 6 : liste des procédures et des fonctions de la base atelier2

3.7 Suppression des procédures et de la fonction

On supprime ensuite deux procédures et la fonction avec `DROP ... IF EXISTS` (le `IF EXISTS` évite une erreur si l'objet n'existe pas). En relançant les deux commandes de la section précédente, on constate qu'il ne reste que `pc1` parmi les procédures, et que la liste des fonctions est vide (« Empty set »).



```
MySQL [atelier2]> DROP PROCEDURE IF EXISTS afficher_stagiaires_par_ville;
Query OK, 0 rows affected (0.057 sec)

MySQL [atelier2]> DROP PROCEDURE IF EXISTS compter_stagiaires;
Query OK, 0 rows affected (0.063 sec)

MySQL [atelier2]> DROP FUNCTION IF EXISTS nombre_stagiaires_par_ville;
Query OK, 0 rows affected (0.402 sec)

MySQL [atelier2]> SHOW PROCEDURE STATUS WHERE Db = DATABASE();
+----------+------+-----------+----------+-------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
| Db       | Name | Type      | Language | Definer     | Modified            | Created             | Security_type | Comment | character_set_client | collation_connection | Database Collation |
+----------+------+-----------+----------+-------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
| atelier2 | pc1  | PROCEDURE | SQL      | avnadmin@% | 2026-11-12 14:28:36 | 2026-11-12 14:28:36 | DEFINER       |         | utf8mb4              | utf8mb4_general_ci   | utf8mb4_0900_ai_ci |
+----------+------+-----------+----------+-------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+--------------------+
1 row in set (0.057 sec)
MySQL [atelier2]> SHOW FUNCTION STATUS WHERE Db = DATABASE();
Empty set (0.057 sec)
```

Figure 7 : suppression des objets puis nouvelle vérification

4. Facturation de l’eau

Cet exemple a deux objectifs : insérer cent factures avec des consommations croissantes, puis calculer automatiquement le montant à payer. Le barème est le suivant : 1 € par m³ jusqu’à 10 m³, 1,5 € par m³ de 11 à 30 m³ et 2 € par m³ à partir de 31 m³.

4.1 Insertion de cent factures

La table `facture` contient l’identifiant, le nom du client et la quantité d’eau. La procédure `inserer_factures` utilise une boucle `WHILE` avec un compteur qui va de 1 à 100. À chaque tour elle insère un client nommé Client1, Client2, etc. (avec `CONCAT`) dont la quantité d’eau est égale au compteur, puis elle ajoute 1 au compteur. Après l’appel, un `SELECT COUNT(*)` confirme que la table contient 100 lignes.

```
MySQL [atelier2]> CREATE TABLE facture (
    ->     id_facture INT AUTO_INCREMENT PRIMARY KEY,
    ->     nom_client VARCHAR(255),
    ->     qte_eau INT
    -> );
Query OK, 0 rows affected (0.063 sec)

MySQL [atelier2]> DELIMITER /
MySQL [atelier2]> CREATE PROCEDURE inserer_factures()
    -> BEGIN
    ->     DECLARE compteur INT DEFAULT 1;
    ->     WHILE compteur <= 100 DO
    ->         INSERT INTO facture (nom_client, qte_eau)
    ->         VALUES (CONCAT('Client', compteur), compteur);
    ->         SET compteur = compteur + 1;
    ->     END WHILE;
    -> END /
Query OK, 0 rows affected (0.071 sec)

MySQL [atelier2]> DELIMITER ;
MySQL [atelier2]> CALL inserer_factures();
Query OK, 1 row affected (0.181 sec)

MySQL [atelier2]> SELECT COUNT(*) FROM facture;
+----------+
| COUNT(*) |
+----------+
|      100 |
+----------+
1 row in set (0.065 sec)
```

Figure 8 : création de la table facture et de la procédure inserer_factures

4.2 La fonction calculer_montant

La fonction reçoit une quantité d'eau et retourne le montant en `DECIMAL(10,2)`. Elle utilise `IF / ELSEIF / ELSE` pour choisir la tranche. Les tranches s'additionnent : au-delà de 30 m³, les 10 premiers m³ sont payés 1 €, les 20 suivants 1,5 € et le reste 2 €.

```
MySQL [atelier2]> CREATE FUNCTION calculer_montant(qte INT)
    -> RETURNS DECIMAL(10,2)
    -> BEGIN
    ->     DECLARE montant DECIMAL(10,2);
    ->     IF qte <= 10 THEN
    ->         SET montant = qte * 1;
    ->     ELSEIF qte <= 30 THEN
    ->         SET montant = (10 * 1) + (qte - 10) * 1.5;
    ->     ELSE
    ->         SET montant = (10 * 1) + (20 * 1.5) + (qte - 30) * 2;
    ->     END IF;
    ->     RETURN montant;
    -> END /
Query OK, 0 rows affected (0.056 sec)
```

Figure 9 : création de la fonction calculer_montant

4.3 Tests de la fonction

On teste d'abord la fonction avec trois valeurs. Pour 5 m³ on obtient 5,00 €. Pour 20 m³ : 10 + 10 × 1,5 = 25,00 €. Pour 40 m³ : 10 + 30 + 10 × 2 = 60,00 €. Ces résultats correspondent à ceux calculés à la main. On applique ensuite la fonction à la table `facture` pour quelques quantités choisies autour des limites des tranches. On voit par exemple que 10 m³ coûtent 10,00 € alors que 11 m³ coûtent 11,50 € (le 11e m³ est facturé 1,5 €), et que 30 m³ coûtent 40,00 € contre 42,00 € pour 31 m³. Pour 100 m³ : 10 + 30 + 70 × 2 = 180,00 €.

```
MySQL [atelier2]> DELIMITER ;
MySQL [atelier2]> SELECT calculer_montant(5), calculer_montant(20), calculer_montant(40);
+---------------------+----------------------+----------------------+
| calculer_montant(5) | calculer_montant(20) | calculer_montant(40) |
+---------------------+----------------------+----------------------+
|                5.00 |                25.00 |                60.00 |
+---------------------+----------------------+----------------------+
1 row in set (0.060 sec)

MySQL [atelier2]> SELECT id_facture, nom_client, qte_eau,
    ->        calculer_montant(qte_eau) AS montant_a_payer
    -> FROM facture
    -> WHERE qte_eau IN (1, 5, 10, 11, 20, 30, 31, 40, 50, 75, 100);
+------------+------------+---------+-----------------+
| id_facture | nom_client | qte_eau | montant_a_payer |
+------------+------------+---------+-----------------+
|          1 | Client1    |       1 |            1.00 |
|          5 | Client5    |       5 |            5.00 |
|         10 | Client10   |      10 |           10.00 |
|         11 | Client11   |      11 |           11.50 |
|         20 | Client20   |      20 |           25.00 |
|         30 | Client30   |      30 |           40.00 |
|         31 | Client31   |      31 |           42.00 |
|         40 | Client40   |      40 |           60.00 |
|         50 | Client50   |      50 |           80.00 |
|         75 | Client75   |      75 |          130.00 |
|        100 | Client100  |     100 |          180.00 |
+------------+------------+---------+-----------------+
11 rows in set (0.053 sec)
```

Figure 10 : appels de calculer_montant, directement puis sur la table facture

5. Les paramètres IN, OUT et INOUT

5.1 Paramètre IN : supprimer_stagiaire

Le paramètre IN sert à envoyer une valeur à la procédure ; c'est le mode par défaut. Ici `supprimer_stagiaire` reçoit un identifiant `p_id` et supprime la ligne correspondante. Avec `CALL supprimer_stagiaire(5);`, le stagiaire Youssef est supprimé (1 row affected), et le `SELECT` qui suit montre qu'il reste quatre stagiaires.

```
MySQL [atelier2]> DELIMITER /
MySQL [atelier2]> CREATE PROCEDURE supprimer_stagiaire(IN p_id INT)
    -> BEGIN
    ->     DELETE FROM stagiaires WHERE id = p_id;
    -> END /
Query OK, 0 rows affected (0.053 sec)

MySQL [atelier2]> DELIMITER ;
MySQL [atelier2]> CALL supprimer_stagiaire(5);
Query OK, 1 row affected (0.054 sec)

MySQL [atelier2]> SELECT * FROM stagiaires;
+----+--------+------------+
| id | nom    | ville      |
+----+--------+------------+
|  1 | Ali    | Casablanca |
|  2 | Sara   | Rabat      |
|  3 | Khalid | Casablanca |
|  4 | Mouna  | Marrakech  |
+----+--------+------------+
4 rows in set (0.056 sec)
```

Figure 11 : procédure avec un paramètre IN

5.2 Paramètre OUT : compter_stagiaires_out

Le paramètre OUT permet de récupérer une valeur calculée par la procédure. On appelle `compter_stagiaires_out` avec une variable utilisateur, `@nb`, que la procédure remplit. Un `SELECT @nb` affiche ensuite 4, valeur cohérente avec la suppression faite juste avant (il y avait 5 stagiaires au début).

```
MySQL [atelier2]> DELIMITER /
MySQL [atelier2]> CREATE PROCEDURE compter_stagiaires_out(OUT total INT)
    -> BEGIN
    ->     SELECT COUNT(*) INTO total FROM stagiaires;
    -> END /
Query OK, 0 rows affected (0.073 sec)

MySQL [atelier2]> DELIMITER ;
MySQL [atelier2]> CALL compter_stagiaires_out(@nb);
Query OK, 1 row affected (0.080 sec)

MySQL [atelier2]> SELECT @nb AS total_stagiaires;
+-----------------+
| total_stagiaires |
+-----------------+
|               4 |
+-----------------+
1 row in set (0.048 sec)
```

Figure 12 : procédure avec un paramètre OUT

5.3 Paramètre INOUT : augmenter_stock

Le paramètre INOUT sert à la fois à envoyer une valeur et à en récupérer une. Cet exemple a besoin d'une table `produits` qui n'existait pas encore : elle a été créée avec les colonnes `id`, `nom` et `stock`, et contient un produit « Stylo » avec un stock de 50. La procédure `augmenter_stock` ajoute la quantité `qte` au stock du produit, puis remet le nouveau stock dans `qte`. La variable `@ajout` est initialisée à 10 avant l'appel. Après `CALL augmenter_stock(@ajout, 1);`, elle vaut 60, c'est-à-dire 50 + 10 : elle est entrée avec 10 et ressortie avec 60.

```
MySQL [atelier2]> CREATE TABLE produits (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     nom VARCHAR(100),
    ->     stock INT
    -> );
Query OK, 0 rows affected (0.068 sec)

MySQL [atelier2]>
MySQL [atelier2]> INSERT INTO produits (nom, stock) VALUES ('Stylo', 50);
Query OK, 1 row affected (0.051 sec)

MySQL [atelier2]>
MySQL [atelier2]> DELIMITER /
MySQL [atelier2]> CREATE PROCEDURE augmenter_stock(INOUT qte INT, IN p_id INT)
    -> BEGIN
    ->     UPDATE produits SET stock = stock + qte WHERE id = p_id;
    ->     SELECT stock INTO qte FROM produits WHERE id = p_id;
    -> END /
Query OK, 0 rows affected (0.053 sec)

MySQL [atelier2]> DELIMITER ;
MySQL [atelier2]>
MySQL [atelier2]> SET @ajout = 10;
Query OK, 0 rows affected (0.352 sec)

MySQL [atelier2]> CALL augmenter_stock(@ajout, 1);
Query OK, 1 row affected (0.098 sec)

MySQL [atelier2]> SELECT @ajout AS nouveau_stock;
+---------------+
| nouveau_stock |
+---------------+
|            60 |
+---------------+
1 row in set (0.047 sec)
```

Figure 13 : procédure avec un paramètre INOUT

5.4 Récapitulatif

Type	Direction	Rôle	Exemple de l'atelier
IN	Entrée	Envoyer une valeur à la procédure	supprimer_stagiaire(5)
OUT	Sortie	Récupérer une valeur calculée	compter_stagiaires_out(@nb)
INOUT	Entrée et sortie	Envoyer une valeur, puis la récupérer modifiée	augmenter_stock(@ajout, 1)

6. Conclusion

Cet atelier a permis de voir comment on crée, on appelle et on supprime des procédures et des fonctions stockées dans MySQL. La différence principale est qu'une fonction retourne une valeur et s'utilise dans un `SELECT`, alors qu'une procédure s'appelle avec `CALL` et peut renvoyer des résultats par ses paramètres OUT et INOUT.

L'exemple de la facturation montre l'intérêt de ces outils : une seule procédure a rempli la table avec cent lignes, et une seule fonction, avec ses conditions, calcule le montant de chaque facture. Les résultats obtenus ont été vérifiés à la main sur plusieurs valeurs.

Le seul point de blocage concerne le délimiteur : sans le changer avec `DELIMITER`, la création d'une procédure échoue à cause des points-virgules. Il faut aussi penser à créer les tables utilisées par les exemples, comme `produits` pour le paramètre INOUT, qui n'était pas prévue dans le sujet.